

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
<i>Weightage</i>	20%	30%	20%	10%	20%	<i>100%</i>
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I PRANEESH.A(192571006) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3

Service Combination	Support Count
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Q4. Dataset: Student Course Enrollment

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Q1. Apriori Algorithm Pseudocode – Online Shopping

Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question: Design pseudocode for the Apriori algorithm to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold.

Answer:

Problem Understanding:

- Input: Transaction database $D = \{O1, O2, O3, O4, O5\}$, minimum support threshold (min_sup)
- Output: All frequent itemsets that meet the minimum support
- The Apriori algorithm uses a level-wise, breadth-first search approach
- Key Principle: Apriori Property – All subsets of a frequent itemset must also be frequent

Pseudocode:

ALGORITHM: Apriori(D, min_sup)

INPUT: D = Transaction Database

min_sup = Minimum Support Threshold

OUTPUT: L = Set of all frequent itemsets

BEGIN

// STEP 1: Generate frequent 1-itemsets

$C_1 \leftarrow \{\text{all individual items in } D\}$

```

FOR EACH candidate c IN C1 DO

    c.count ← COUNT of transactions in D containing c

END FOR

L1 ← {c ∈ C1 | c.count / |D| ≥ min_sup}


// STEP 2: Iteratively generate larger itemsets

k ← 2

WHILE L{k-1} ≠ ∅ DO


    // CANDIDATE GENERATION (Join Step)

    Ck ← ∅

    FOR EACH pair (l1, l2) IN L{k-1} × L{k-1} DO

        IF l1 and l2 share first (k-2) items THEN

            candidate ← l1 ∪ l2

            // PRUNING STEP

            IF all (k-1)-subsets of candidate ∈ L{k-1} THEN

                ADD candidate TO Ck

            END IF

        END IF

    END FOR

    END FOR


    // SUPPORT COUNTING (Database Scan)

    FOR EACH transaction t IN D DO

        FOR EACH candidate c IN Ck DO

            IF c ⊆ t THEN

                c.count ← c.count + 1

            END IF

        END FOR

    END FOR

    END FOR


    // FILTER by minimum support

```

$$L_k \leftarrow \{c \in C_k \mid c.\text{count} / |D| \geq \text{min_sup}\}$$

$$k \leftarrow k + 1$$

END WHILE

// STEP 3: Return all frequent itemsets

$$\text{RETURN } L \leftarrow L_1 \cup L_2 \cup \dots \cup L_{\{k-1\}}$$

END

Illustration with Given Dataset (min_sup = 40%, i.e., 2 transactions):

Iteration 1 – Scan Database, Count Items:

Item	Count	Support	Frequent?
Mobile	4	80%	✓
Earphones	3	60%	✓
Charger	3	60%	✓

$L_1 = \{\text{Mobile, Earphones, Charger}\}$

Iteration 2 – Join L_1 to form C_2 , Count:

Candidate	Count	Support	Frequent?
{Mobile, Earphones}	3 (O1,O4,O5)	60%	✓
{Mobile, Charger}	2 (O2,O4)	40%	✓
{Earphones, Charger}	2 (O3,O4)	40%	✓

$L_2 = \{\text{Mobile, Earphones}\}, \{\text{Mobile, Charger}\}, \{\text{Earphones, Charger}\}$

Iteration 3 – Join L_2 to form C_3 :

- $\{\text{Mobile, Earphones}\} \cup \{\text{Mobile, Charger}\} = \{\text{Mobile, Earphones, Charger}\}$
- All 2-subsets are in L_2 , so candidate is valid

Candidate	Count	Support	Frequent?
{Mobile, Earphones, Charger}	1 (O4)	20%	X

$L_3 = \emptyset$. Algorithm terminates.

Completeness: The algorithm handles all cases including empty frequent sets, single-item databases, and terminates when no more frequent itemsets can be generated.

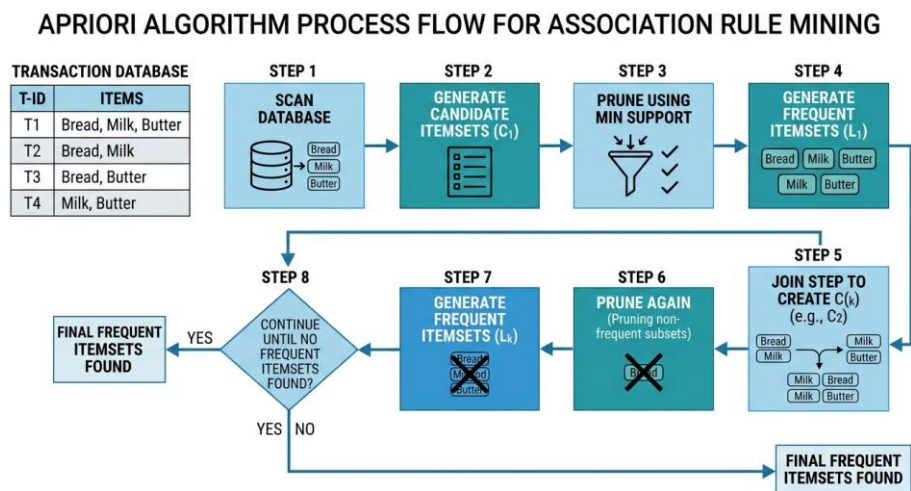


Figure 6: Apriori Algorithm Process Flow Diagram

Q2. FP-Growth Algorithm Pseudocode – Library Records

Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question: Develop pseudocode for the FP-Growth algorithm to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree.

Answer:

Problem Understanding:

- FP-Growth (Frequent Pattern Growth) mines frequent itemsets WITHOUT candidate generation
- It compresses the database into an FP-Tree (Frequent Pattern Tree)
- Uses divide-and-conquer strategy to mine patterns from the tree
- Much faster than Apriori for large datasets as it requires only 2 database scans

Pseudocode – Part 1: FP-Tree Construction:

```
ALGORITHM: ConstructFPTree(D, min_sup)
```

```
INPUT: D = Transaction Database
```

```
    min_sup = Minimum Support Threshold
```

```
OUTPUT: FP-Tree with Header Table
```

```
BEGIN
```

```
    // PASS 1: Scan database to find frequent items
```

```
    freq_items ← {}
```

```
    FOR EACH transaction t IN D DO
```

```

FOR EACH item i IN t DO
    freq_items[i] ← freq_items[i] + 1
END FOR

END FOR

// Remove infrequent items and sort by frequency
F_list ← {i ∈ freq_items | freq_items[i] ≥ min_sup}
SORT F_list in descending order of frequency

// Initialize FP-Tree
CREATE root node (labeled "null")
CREATE header_table with entries for each item in F_list

// PASS 2: Build the FP-Tree
FOR EACH transaction t IN D DO
    // Filter and sort transaction items
    sorted_items ← items in t that are in F_list,
                    sorted by F_list order

    // Insert into tree
    current_node ← root
    FOR EACH item IN sorted_items DO
        IF current_node has child labeled item THEN
            child.count ← child.count + 1
            current_node ← child
        ELSE
            CREATE new_node with label = item, count = 1
            ADD new_node as child of current_node
            LINK new_node to header_table entry for item
            current_node ← new_node
        END IF
    END FOR
END FOR

```

```

        END FOR

    END FOR

    RETURN FP-Tree, header_table

END

```

Pseudocode – Part 2: FP-Growth Mining:

ALGORITHM: FPGrowth(FP_Tree, header_table, prefix, min_sup)

INPUT: FP_Tree, header_table, prefix (initially empty)

min_sup = Minimum Support Threshold

OUTPUT: All frequent itemsets

BEGIN

// Process each item in header table (bottom-up)

FOR EACH item IN header_table (ascending frequency) DO

// Generate frequent pattern

new_pattern $\leftarrow$ prefix $\cup$ {item}

new_pattern.support $\leftarrow$ item.support_count

OUTPUT new_pattern as frequent itemset

// Construct Conditional Pattern Base

cond_pattern_base $\leftarrow$ {}

FOR EACH path from item's node to root DO

prefix_path $\leftarrow$ items along the path (excluding item)

cond_pattern_base $\leftarrow$ cond_pattern_base $\cup$

{(prefix_path, item_node.count)}

END FOR

```

// Build Conditional FP-Tree

cond_FP_Tree ← ConstructFPTree(cond_pattern_base, min_sup)

// Recursive mining

IF cond_FP_Tree is not empty THEN

    FPGrowth(cond_FP_Tree, cond_header_table,
              new_pattern, min_sup)

END IF

END FOR

END

```

FP-Tree Construction Steps for Given Dataset (min_sup = 2):

Pass 1 – Item Frequencies:

Item	Count	Frequent?
Python	4	✓ (most frequent)
Machine Learning	3	✓
Data Science	3	✓

Sorted F-list: Python(4) > Machine Learning(3) > Data Science(3)

Pass 2 – Sorted Transactions and Tree Insertion:

Borrow ID	Original	Sorted (by F-list)
B1	Data Science, Python	Python, Data Science
B2	Python, Machine Learning	Python, Machine Learning
B3	Data Science, Machine Learning	Machine Learning, Data Science
B4	Data Science, Python, ML	Python, Machine Learning, Data Science
B5	Python, Machine Learning	Python, Machine Learning

Tree Construction Step-by-Step:

After B1: null → Python:1 → Data Science:1

After B2: null → Python:2 → Data Science:1, Python:2 → ML:1

After B3: null → ML:1 → Data Science:1 (new branch)

After B4: null → Python:3 → ML:2 → Data Science:1

After B5: null → Python:4 → ML:3

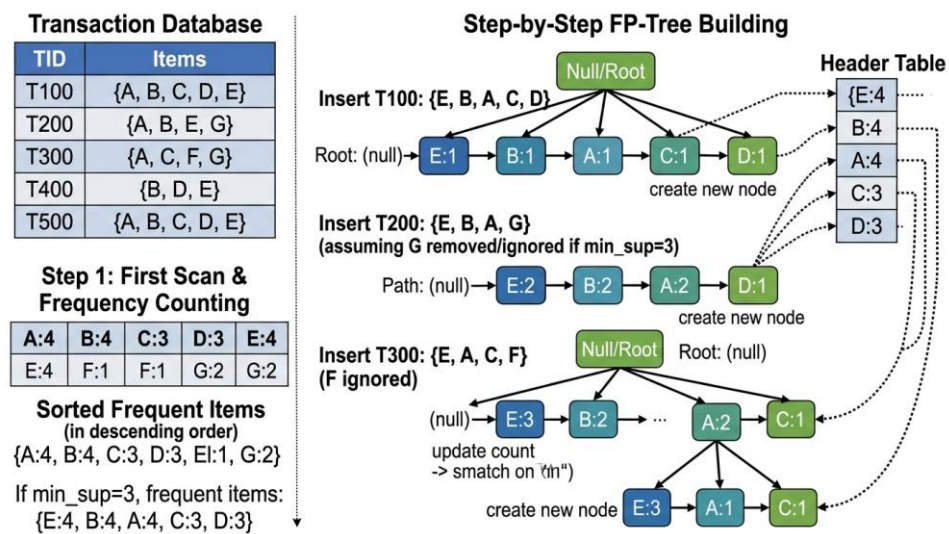


Figure 7: FP-Tree Construction Process

Q3. Association Rule Generation Pseudocode – Service Combinations

Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question: Write pseudocode to generate association rules from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold.

Answer:

Problem Understanding:

- Input: Pre-computed frequent itemsets with support counts, total transactions N, minimum confidence threshold
- Output: Valid association rules with support and confidence values
- For each frequent itemset, generate all possible non-empty proper subsets and create rules

Pseudocode:

ALGORITHM: GenerateAssociationRules(L, min_conf, N)

INPUT: L = Set of frequent itemsets with support counts

min_conf = Minimum Confidence Threshold

N = Total number of transactions

OUTPUT: R = Set of valid association rules

BEGIN

R $\leftarrow \emptyset$ // Set of valid rules

```

// STEP 1: Process each frequent itemset of size  $\geq 2$ 

FOR EACH frequent itemset  $f$  IN  $L$  WHERE  $|f| \geq 2$  DO

    // STEP 2: Generate all non-empty proper subsets

    subsets  $\leftarrow$  GenerateAllSubsets( $f$ )

    FOR EACH subset  $s$  IN subsets DO

        // STEP 3: Create rule:  $s \rightarrow (f - s)$ 

        antecedent  $\leftarrow s$ 

        consequent  $\leftarrow f - s$  // set difference

        IF consequent  $\neq \emptyset$  THEN

            // STEP 4: Calculate Confidence

            confidence  $\leftarrow$  support_count( $f$ ) / support_count( $s$ )

            // STEP 5: Calculate Support

            support  $\leftarrow$  support_count( $f$ ) /  $N$ 

            // STEP 6: Apply Confidence Filter

            IF confidence  $\geq$  min_conf THEN

                // STEP 7: Calculate Lift (optional)

                lift  $\leftarrow$  confidence / (support_count(consequent) /  $N$ )

                // Create and store the rule

                rule  $\leftarrow$  {

                    antecedent:  $s$ ,

                    consequent: consequent,

                    support: support,

                    confidence: confidence,

                    lift: lift

                }

```

```

        ADD rule TO R
    END IF
END IF
END FOR
END FOR

RETURN R
END

// Helper Function: Generate all non-empty proper subsets
FUNCTION GenerateAllSubsets(itemset)

    subsets  $\leftarrow \emptyset$ 

    n  $\leftarrow$  |itemset|

    // Use bit masking: from 1 to  $2^n - 2$ 
    FOR i  $\leftarrow$  1 TO ( $2^n - 2$ ) DO
        subset  $\leftarrow \{\}$ 
        FOR j  $\leftarrow$  0 TO n-1 DO
            IF bit j of i is set THEN
                ADD itemset[j] TO subset
            END IF
        END FOR
        ADD subset TO subsets
    END FOR

    RETURN subsets
END

```

Application to Given Dataset (min_conf = 60%, N = 5):

Processing {Internet, Streaming} (support count = 3):

Rule	Confidence Calc	Confidence	Lift	Valid?
Internet → Streaming	3/4	75%	$75\%/(4/5)=0.94$	✓ Yes
Streaming → Internet	3/4	75%	$75\%/(4/5)=0.94$	✓ Yes

Processing {Streaming, Cloud Storage} (support count = 2):

Rule	Confidence Calc	Confidence	Lift	Valid?
Streaming → Cloud Storage	2/4	50%	$50\%/(3/5)=0.83$	✗ No
Cloud Storage → Streaming	2/3	66.7%	$66.7\%/(4/5)=0.83$	✓ Yes

Final Valid Rules:

#	Rule	Support	Confidence	Lift
1	Internet → Streaming	60%	75%	0.94
2	Streaming → Internet	60%	75%	0.94
3	Cloud Storage → Streaming	40%	66.7%	0.83

Q4. Constraint-Based Frequent Itemset Mining Pseudocode

Dataset: Student Course Enrollment

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question: Design pseudocode for constraint-based frequent itemset mining where only itemsets containing the course "Python" are considered during the mining process.

Answer:

Problem Understanding:

- Constraint: Every frequent itemset must contain "Python" (succinct anti-monotone constraint)
- This constraint can be pushed into the mining process (not just post-filtering)
- Pushing constraints deep into mining significantly reduces the search space

Pseudocode:

ALGORITHM: ConstraintBasedMining(D, min_sup, constraint_item)

INPUT: D = Transaction Database

min_sup = Minimum Support Threshold

constraint_item = Required item ("Python")

OUTPUT: L = Frequent itemsets containing constraint_item

BEGIN

// STEP 1: Check if constraint item is frequent

count_constraint ← COUNT transactions containing constraint_item

IF count_constraint / |D| < min_sup THEN

```

    RETURN  $\emptyset$  // Constraint item itself not frequent
END IF

// STEP 2: Filter transactions to only those containing
//     the constraint item
D_filtered  $\leftarrow \{\}$ 
FOR EACH transaction t IN D DO
    IF constraint_item  $\in$  t THEN
        ADD t TO D_filtered
    END IF
END FOR

// STEP 3: Find frequent 1-itemsets from filtered data
C1  $\leftarrow$  {all individual items in D_filtered}
FOR EACH candidate c IN C1 DO
    c.count  $\leftarrow$  COUNT of transactions in D containing c
END FOR
L1  $\leftarrow$  {c  $\in$  C1 | c.count / |D|  $\geq$  min_sup}

// Ensure constraint_item is in L1
IF constraint_item  $\notin$  L1 THEN
    RETURN  $\emptyset$ 
END IF

// STEP 4: Level-wise mining with constraint enforcement
k  $\leftarrow$  2
WHILE L_{k-1}  $\neq \emptyset$  DO

    // CANDIDATE GENERATION with constraint
    C_k  $\leftarrow \emptyset$ 

    FOR EACH pair (l1, l2) IN L_{k-1}  $\times$  L_{k-1} DO

```

```

candidate  $\leftarrow$   $l_1 \cup l_2$ 

// CONSTRAINT CHECK: Must contain constraint_item

IF constraint_item  $\notin$  candidate THEN

    SKIP candidate // Prune immediately

    CONTINUE

END IF

// STANDARD PRUNING: Anti-monotone property

IF all (k-1)-subsets of candidate  $\in L_{k-1}$  THEN

    ADD candidate TO  $C_k$ 

END IF

END FOR

// SUPPORT COUNTING

FOR EACH transaction t IN D DO

    FOR EACH candidate c IN  $C_k$  DO

        IF  $c \subseteq t$  THEN

            c.count  $\leftarrow$  c.count + 1

        END IF

    END FOR

END FOR

 $L_k \leftarrow \{c \in C_k \mid \text{c.count} / |D| \geq \text{min\_sup}\}$ 

k  $\leftarrow$  k + 1

END WHILE

RETURN  $L \leftarrow L_1 \cup L_2 \cup \dots \cup L_{k-1}$ 

(filtered to only include sets with constraint_item)

END

```

Illustration with Dataset (min_sup = 40%, constraint = Python):

Step 1: Python appears in S1, S2, S4, S5 → count = 4, support = 80% ✓

Step 2: Filtered transactions: {S1, S2, S4, S5}

Frequent 1-itemsets from filtered data (counted against full D):

Item	Count in D	Support	Frequent?
Python	4	80%	✓
Database	3	60%	✓
AI	3	60%	✓

Constrained 2-itemsets (must contain Python):

Candidate	Count	Support	Frequent?
{Python, Database}	2 (S1, S4)	40%	✓
{Python, AI}	2 (S2, S4)	40%	✓
{Database, AI} — PRUNED	—	—	Skipped (no Python)

The constraint pruned {Database, AI} from candidate generation, saving one support counting operation.

Constrained 3-itemsets:

Candidate	Count	Support	Frequent?
{Python, Database, AI}	1 (S4)	20%	X

How the Constraint Affects Candidate Generation and Pruning:

- 1. Early Pruning: {Database, AI} is eliminated from C_2 without ever computing its support, saving one database scan per eliminated candidate.
- 2. Reduced Search Space: Without constraint, $C_2 = 3$ candidates. With constraint, $C_2 = 2$ candidates (33% reduction).

- 3. Transaction Filtering: By pre-filtering to Python-containing transactions (Step 2), the effective database size decreases from 5 to 4, speeding up support counting.
- 4. Succinctness: The constraint 'must contain Python' is a succinct constraint—it can be directly used to enumerate candidate itemsets rather than generating all candidates and filtering afterward.

Q5. Multilevel Association Rule Mining Pseudocode – Hospital

Dataset: Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question: Develop pseudocode for multilevel association rule mining using the treatment hierarchy. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Answer:

Problem Understanding:

Treatment Hierarchy:

Healthcare (Level 0)

├── Cardiology (Level 1)

│ ├── ECG (Level 2)

└── Orthopedics (Level 1)

- Mining at different hierarchy levels reveals patterns at varying granularities
- Higher levels: more general, higher support
- Lower levels: more specific, lower support, potentially more actionable

Pseudocode:

ALGORITHM: MultilevelAssociationMining(D, H, min_sup[], min_conf)

INPUT: D = Transaction Database

H = Concept Hierarchy (tree structure)

min_sup[] = Array of minimum support per level

min_conf = Minimum Confidence Threshold

OUTPUT: R = Set of association rules at each level

BEGIN

// STEP 1: Determine hierarchy levels

max_level $\leftarrow$ GetMaxDepth(H)

R $\leftarrow \emptyset$

// STEP 2: Mine rules at each level (top-down)

FOR level $\leftarrow$ 0 TO max_level DO

// STEP 3: Encode transactions at current level

D_level $\leftarrow \emptyset$

FOR EACH transaction t IN D DO

encoded_t $\leftarrow \emptyset$

FOR EACH item i IN t DO

ancestor $\leftarrow$ GetAncestorAtLevel(i, H, level)

IF ancestor $\neq$ NULL THEN

ADD ancestor TO encoded_t

END IF

END FOR

ADD encoded_t TO D_level

END FOR

// STEP 4: Apply Apriori at current level

L_level $\leftarrow$ Apriori(D_level, min_sup[level])

// STEP 5: Generate rules from frequent itemsets

FOR EACH frequent itemset f IN L_level DO

rules $\leftarrow$ GenerateRules(f, min_conf)


```

// STEP 6: Redundancy Check

FOR EACH rule r IN rules DO
    IF NOT IsRedundant(r, R) THEN
        r.level  $\leftarrow$  level
        ADD r TO R
    END IF
END FOR

END FOR

RETURN R

END

// Redundancy Check Function

FUNCTION IsRedundant(new_rule, existing_rules)
    FOR EACH rule r IN existing_rules DO
        IF r is ancestor of new_rule in hierarchy
            AND r.support  $\approx$  new_rule.support
            AND r.confidence  $\approx$  new_rule.confidence THEN
                RETURN TRUE // Redundant
            END IF
    END FOR
    RETURN FALSE
END

```

Rules at Different Levels:

Level 0 Rules (General):

Rule	Support	Confidence	Usefulness
Healthcare → Healthcare	5/5=100%	100%	Trivial – not useful

Level 1 Rules (Department):

Rule	Support	Confidence	Usefulness
Healthcare → Cardiology	3/5=60%	3/5=60%	Moderately useful
Healthcare → Orthopedics	2/5=40%	2/5=40%	Moderately useful

Level 2 Rules (Specific Treatment):

Rule	Support	Confidence	Usefulness
Cardiology → ECG	2/5=40%	2/3=66.7%	Highly useful
ECG → Cardiology	2/5=40%	2/2=100%	Highly useful

How Concept Hierarchies Improve Pattern Discovery:

- 1. Flexible Granularity: Higher levels capture general trends (e.g., most patients need Cardiology), while lower levels reveal specific patterns (e.g., Cardiology patients need ECG).
- 2. Overcoming Low Support: An item like ECG might not meet support at the general level, but at the Cardiology level, it's very frequent. Variable support thresholds per level address this.
- 3. Actionable Rules: Level 2 rule 'Cardiology → ECG' (67% confidence) is directly actionable—hospitals can pre-schedule ECG tests for Cardiology admissions.
- 4. Redundancy Elimination: Without hierarchy awareness, both 'Healthcare → ECG' and 'Cardiology → ECG' would be generated. The redundancy check ensures only the most specific, informative rule is kept.

